Large Language Models: Architectures, Training Paradigms, and Open Challenges

A. Sample Author

Department of Computer Science, Example Institute of Technology

Abstract. Large language models (LLMs) have become the dominant paradigm in natural language processing and are increasingly used as general-purpose reasoning engines. This paper surveys the architectural foundations of modern LLMs, from the original Transformer to decoder-only and mixture-of-experts designs, and reviews the pre-training, instruction tuning, and preference-alignment pipelines used to produce assistant-style models. We summarize empirical scaling behavior, discuss evaluation practices, and highlight open problems including hallucination, efficiency, and safety. The document is intended as an illustrative sample of a short technical paper.

Keywords: large language models, Transformer, scaling laws, alignment, retrieval-augmented generation

1. Introduction

Language modeling, the task of predicting the next token given a sequence of preceding tokens, has existed for decades. What changed in the past several years is scale: models with billions to trillions of parameters, trained on trillions of tokens, display capabilities such as in-context learning, code generation, and multi-step reasoning that were not explicitly programmed. These systems, commonly called large language models, now underpin chat assistants, coding tools, search products, and scientific workflows.

This paper provides a compact overview of how LLMs work and how they are built. Section 2 describes the Transformer architecture and its main variants. Section 3 covers training, including pre-training, fine-tuning, and alignment. Section 4 reviews scaling behavior and evaluation. Section 5 discusses limitations and open challenges, and Section 6 concludes.

2. Model Architectures

2.1 The Transformer

The Transformer, introduced by Vaswani et al. [1], replaced recurrence with self-attention, allowing every token to attend directly to every other token in a sequence. Given queries Q, keys K, and values V, scaled dot-product attention is computed as:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(Q K^{T} / \sqrt{d_k} \right) V$$

where d_k is the key dimension. Multi-head attention runs several such operations in parallel on different learned projections, and the outputs are concatenated. Each Transformer block combines attention with a position-wise feed-forward network, residual connections, and layer normalization. Because attention is permutation-invariant, positional information is injected through sinusoidal encodings, learned embeddings, or rotary position embeddings (RoPE).

2.2 Encoder, Decoder, and Encoder-Decoder Models

Three families dominate the literature. Encoder-only models such as BERT [2] are trained with masked language modeling and excel at understanding tasks like classification. Encoder-decoder models such as T5 [3] frame every task as text-to-text. Decoder-only models, exemplified by the GPT series [4], are trained autoregressively and have become the default for generative LLMs because they scale cleanly and support open-ended generation and in-context learning.

2.3 Mixture-of-Experts and Efficiency

Dense models activate all parameters for every token, so compute grows linearly with model size. Mixture-of-experts (MoE) architectures replace the feed-forward layer with many expert networks and a

learned router that selects a small subset per token. This decouples total parameter count from per-token compute, enabling larger capacity at similar inference cost, at the price of added routing complexity and memory requirements. Complementary techniques include grouped-query attention, FlashAttention-style kernels, quantization, and speculative decoding, all of which reduce latency and memory use during serving.

Table 1. Representative model families and their characteristics.

Family	Type	Training objective	Typical use
BERT	Encoder-only	Masked LM	Classification, retrieval
T5	Encoder-decoder	Span corruption	Translation, summarization
GPT-style	Decoder-only	Next-token prediction	Chat, code, reasoning
MoE LLMs	Decoder-only (sparse)	Next-token prediction	Large-scale assistants

3. Training Paradigms

3.1 Pre-training

Pre-training exposes the model to large text corpora drawn from web pages, books, code repositories, and scientific articles. Data quality matters as much as quantity: deduplication, language and quality filtering, and careful mixing of domains consistently improve downstream performance. Tokenization, typically byte-pair encoding or a similar subword scheme, converts text into discrete units. The model minimizes the cross-entropy loss over next-token predictions:

$$L(\theta) = - \sum_{t} \log p_{\theta}(x_{t} \mid x_{<t})$$

3.2 Instruction Tuning and Alignment

A pre-trained model is a powerful text predictor but not necessarily a helpful assistant. Supervised fine-tuning (SFT) on curated instruction-response pairs teaches the model to follow directions. Preference-based methods then refine behavior further. Reinforcement learning from human feedback (RLHF) [5] trains a reward model on human comparisons and optimizes the policy against it, while direct preference optimization (DPO) [6] reaches a similar goal without an explicit reward model or on-policy sampling. More recent approaches use AI-generated feedback guided by written principles, and reinforcement learning with verifiable rewards to improve reasoning on tasks such as mathematics and programming.

3.3 Retrieval and Tool Use

Parametric knowledge stored in model weights is static and imperfect. Retrieval-augmented generation (RAG) [7] addresses this by fetching relevant documents at inference time and conditioning the model on them, improving factual grounding and allowing knowledge updates without retraining. Related work equips models with tools such as code interpreters, search engines, and APIs, turning an LLM into an agent that plans, acts, and observes results over multiple steps.

4. Scaling Behavior and Evaluation

4.1 Scaling Laws

Kaplan et al. [8] showed that test loss decreases as a smooth power law in model size, dataset size, and compute. Hoffmann et al. [9] refined this, arguing that many early large models were undertrained and that, for a fixed compute budget, parameters and training tokens should be scaled in roughly equal proportion. A commonly used approximation is:

$$L(N, D) = E + A / N^{\alpha} + B / D^{\beta}$$

where N is the parameter count, D the number of training tokens, and E the irreducible loss. These relationships guide budget allocation, although they describe loss rather than specific capabilities, some of which appear to emerge abruptly or depend on how they are measured.

4.2 Evaluation

Evaluating LLMs is difficult. Static benchmarks such as MMLU, GSM8K, and HumanEval measure knowledge, mathematical reasoning, and code synthesis, but they are vulnerable to data contamination and saturation. Human preference studies and pairwise comparison platforms capture qualities that automated metrics miss, while LLM-as-a-judge methods offer scalable approximations with known biases such as favoring longer answers. Robust evaluation therefore combines multiple benchmarks, held-out private test sets, and task-specific human review.

Table 2. Common evaluation benchmarks and what they measure.

Benchmark	Capability measured	Format
MMLU	Broad academic knowledge	Multiple choice
GSM8K	Grade-school math reasoning	Free-form numeric answer
HumanEval	Python code synthesis	Unit-test pass rate
TruthfulQA	Resistance to common falsehoods	Generation / multiple choice

5. Limitations and Open Challenges

Hallucination. LLMs can generate fluent but false statements because they optimize for plausible continuations, not verified truth. Retrieval, citation requirements, and calibrated uncertainty estimates reduce but do not eliminate the problem.

Reasoning and reliability. Chain-of-thought prompting and test-time compute scaling improve multi-step reasoning, yet models can still fail on simple variants of problems they solve in other forms, revealing brittle generalization.

Efficiency and cost. Training frontier models requires enormous energy and capital. Research into smaller distilled models, sparse architectures, and low-precision inference aims to make capable models accessible on modest hardware.

Safety, privacy, and bias. Models inherit biases from training data, may memorize sensitive text, and can be misused. Red-teaming, interpretability research, and transparent documentation such as model cards are important complements to alignment training.

Interpretability. Understanding how internal representations give rise to behavior remains an open scientific question. Mechanistic approaches attempt to identify circuits and features, but scaling these methods to the largest models is still challenging.

6. Conclusion

Large language models combine a simple objective, next-token prediction, with scale, data, and careful post-training to yield remarkably general systems. Progress is now driven as much by data curation, alignment methods, and inference-time techniques as by raw parameter count. Key future directions include reliable reasoning, grounded and verifiable outputs, multimodal integration, and efficient deployment. Addressing these challenges will determine how safely and broadly these models can be used.

References

[1] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.

[2] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL, 2019.

[3] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," Journal of Machine Learning Research, 2020.

[4] T. Brown et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.

[5] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," NeurIPS, 2022.

[6] R. Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," NeurIPS, 2023.

[7] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.

[8] J. Kaplan et al., "Scaling Laws for Neural Language Models," arXiv:2001.08361, 2020.

[9] J. Hoffmann et al., "Training Compute-Optimal Large Language Models," arXiv:2203.15556, 2022.